

AR Web Scanner Complete Client Guide

Guide edition: 1.1 Date: 28 August 2026 Audience: Authorized AR Web Scanner operators, Bot Job designers, and test analysts Product scope: Delivered Banca Stato desktop edition, Java V1 execution path Classification: Client operations guide; use only with authorized test data

This guide is based on the delivered React frontend at commit 6270db9 and backend/deployment assets at commit 630410a. Controls and behavior were verified from current source. Live screenshot capture is pending because the configured database contains real client data; every planned figure is therefore shown as a clearly marked capture slot rather than a fabricated image.

Table of contents

- 1 Purpose and safe use
- 2 Interface fundamentals
- 3 Part 1 — Main Dashboard and setup
- 4 Part 2 — Bot Job authoring and execution
- 5 Part 3 — Page Scanner, locators, and OCR
- 6 Part 4 — Data, variables, memory, and output files
- 7 Part 5 — Troubleshooting and quick reference

1. Purpose and safe use

AR Web Scanner helps an authorized user define browser automation as Bot Jobs, organize instructions into Blocks, scan a current browser page for web elements, connect variables, and execute or test the result. The delivered edition opens major tools as independent AR Web pages so that the Bot Job, Page Scanner, Excel Data, Runtime Variables, and output-file state can remain visible together.

Use only approved environments and data. Never place production passwords, account numbers, license content, or private URLs in documentation screenshots. A disabled control normally means that a Bot Job, browser, WebSocket connection, selection, or completed validation is required first.

1.1 Recommended operating sequence

- 1 Confirm the correct Organization and Environment.
- 2 Select or create the Bot Job.
- 3 Open Bot Job and review Blocks, instructions, Web Elements, and variable connections.
- 4 Open Excel Data and choose Real Data or Synthetic Data for testing.
- 5 Use CHECK before Test run or Launch.
- 6 Open Page Scanner when web elements or locators need inspection.
- 7 Use Review Locator when a web element is ambiguous, changed, or cannot be found.
- 8 Monitor Variables, ExcelWriter Manager, and Pages Open when the workflow uses them.

2. Interface fundamentals

2.1 Detached pages and Pages Open

Most workspaces are separate windows. The Pages (x) control opens Pages Open, where a row brings a page to the foreground and X closes that page. Closing the Main Dashboard is different: after confirmation, it closes every AR Web page and stops the backend.

2.2 Status messages

The header status is the first place to check after an action. It reports ready, waiting, success, warning, disconnected, and error states. A timeout or disconnected status means the client did not receive the expected acknowledgement; avoid repeating a destructive action until the page has refreshed and the result is known.

2.3 Grey action controls

The delivered interface uses compact grey controls for routine actions. Text labels such as TEST INPUT, TEST CLICK, RENAME, EDIT CMD, and ROLLBACK describe the action directly. Buttons remain disabled when their authoritative connection or owner state is unavailable.

2.4 Find, search, and selection

Find: filters a displayed grid without changing saved data. Page Scanner Search by: changes what the next scan requests. Checkboxes select rows or Blocks for a later explicit action; selection alone does not delete anything.

2.5 Destructive confirmations

Deletion and database replacement actions display confirmation dialogs. Read the target name and count before confirming. Cancel leaves the current data unchanged.

Part 1 — Main Dashboard and setup

3.1 Main Dashboard

Entry path: Start the authorized AR Web Scanner application. Purpose: Select a Bot Job and open every main client workspace.

Figure 01 — Main Dashboard. Screenshot pending: capture with synthetic Bot Jobs and no client data.

Control	What it does	Important condition or result
Organizations	Opens Organizations and its child Environments.	Available without selecting a Bot Job.
New Bot Job	Opens the New Bot Job workflow.	Requires a valid Organization Environment before creation.
Clone Job	Copies the selected Bot Job into a selected target context.	Enabled after selecting one Bot Job.
Config	Opens Configuration.	Database actions can be destructive.
Info	Opens About this Software.	Includes the path to License Manager.
Launch	Starts the selected Bot Job through the delivered Java V1 execution engine.	Enabled only for a selected launchable Bot Job.
Open Job	Opens Bot Job Details for the selected row.	Double-clicking a row also opens it.
Refresh	Reloads the Bot Job list.	Use after creation, clone, import, or external changes.
Find:	Filters Bot Jobs by visible fields.	Does not change database data.
Row checkbox / header checkbox	Selects one or all loaded Bot Jobs.	Used by ALL (x) deletion.
ALL (x)	Requests deletion of checked Bot Jobs.	Confirmation identifies the count and targets.
Row X	Requests deletion of one Bot Job.	Removes job data and saved components after confirmation.
User menu	Shows the licensed identity and opens Pages Open.	The license status is informational.
Exit	Requests application shutdown.	Closes child pages and backend after confirmation/processing.

Normal workflow

- 1 Click a Bot Job row.
- 2 Confirm that Open Job, Clone Job, and Launch become available as appropriate.
- 3 Click Open Job to inspect and test the automation before a normal launch.

Caution: Launch executes the job. Use CHECK and a safe environment before running a workflow that submits data or changes an external system.

3.2 Organizations and Environments

Entry path: Main Dashboard → Organizations. Purpose: Maintain the owner and URL contexts used by Bot Jobs.

Figure 02 — Organizations and child Environments. Screenshot pending.

Control	What it does	Important condition or result
Organization	Selects an existing Organization or + New Organization.	Populates the Organization form and Environment list.
Environment	Selects an existing Environment or + New Environment.	Disabled until an Organization exists.
Organization Name	Defines the displayed owner name.	Required for a useful Organization record.

Control	What it does	Important condition or result
URL Baseline	Stores the Organization's baseline URL.	Use an approved non-secret URL.
Advanced Organization fields	Store priority, search, and option configuration.	Use only values agreed for the client environment.
Create Organization / Update Organization	Creates or updates the displayed Organization.	The label changes according to whether an ID exists.
Delete	Deletes an Organization that has no active Bot Jobs.	Backend refuses an in-use Organization.
Environment Name	Names a child environment such as TEST or UAT.	Stored under the selected Organization.
Environment URL	Defines the exact starting URL.	Validate before creating Bot Jobs.
Create Environment / Update Environment	Creates or updates the displayed Environment.	Requires an Organization.
Delete Environment	Deletes the selected child Environment.	Disabled when deletion would violate the current rules.

Caution: Deleting an Organization or Environment is persistent. The confirmation explains that in-use records are refused.

3.3 New Bot Job

Entry path: Main Dashboard → New Bot Job. Purpose: Create an empty automation job owned by one Organization and Environment.

Figure 03 — New Bot Job. Screenshot pending.

- 1 Select an Organization in the left grid.
- 2 Select one of its Environments in the right grid.
- 3 Enter Bot Job Name and optional Description.
- 4 Choose Web App, Android, or iOS under Application Type.
- 5 Confirm the Organization, Environment, and URL summary.
- 6 Click Create Bot Job.

Control	What it does
Organizations / Environments	Opens the maintenance page for missing owner data.
Refresh	Reloads available Organizations and Environments.
Find: in either grid	Filters only that selection grid.
Cancel / Close	Leaves without creating the job.
Create Bot Job	Creates the record when name and Environment are valid.

3.4 Clone Job

Entry path: Select a Bot Job on Main Dashboard → Clone Job. Purpose: Copy a source Bot Job while assigning a new name, owner context, URL, and application type.

Figure 04 — Clone Job. Screenshot pending.

Review the read-only Source Bot Job summary. Enter New Bot Job Name, Description, and Target URL, then select Application Type, target Organization, and target Environment. Click Clone Bot Job only after the summary identifies the intended target.

Caution: Cloning copies the source job structure and related records. A partial database failure is reported and should not be assumed successful.

3.5 Configuration

Entry path: Main Dashboard → Config. Purpose: Select browser and database type, set operational paths, and perform authorized database maintenance.

Figure 05 — Configuration. Screenshot pending.

Control	What it does	Caution
Browser	Chooses the shared Playwright browser.	Changing an active browser can require closing and replacement.
DB Type	Selects the configured database type.	The delivered client uses path-based Access/SQLite configurations.
Operational path fields and ...	Edit or browse for supported database/report paths.	Use absolute authorized paths.
Reload Configs	Saves/reloads the selected database connection.	Successful reload closes other pages so they cannot retain stale owners.
Backup DB	Chooses a destination and creates a database backup after confirmation.	Verify the destination folder.
Restore DB	Restores data for Date Restore after confirmation.	Replaces current data and closes other pages.
Delete DB	Deletes all Bot Job details in the selected database.	Destructive and irreversible; confirm the database type.
Organizations	Opens Organization maintenance.	Useful after a reload.

3.6 About this Software

Entry path: Main Dashboard → Info. Purpose: Display installed software and license information.

Figure 06 — About this Software. Screenshot pending.

Use Refresh to reload displayed information. Open License Manager and Manage License both open the independent License Request page. Pages (x) opens the page manager, and Close closes only the About page.

3.7 License Request

Entry path: About this Software → Open License Manager or Manage License. Purpose: Generate a license request, activate a response, or select an existing license file.

Figure 07 — License Request. Screenshot pending; never capture license content.

Mode/control	What it does
Request	Requires Organization, Owner, Email, and agreement acceptance; Generate request creates the request.
Activate	Selects a response file and uses Activate license.
Use existing	Selects an existing license file and uses Use existing license.
Refresh license status	Reloads the current validity state.
Agreement checkbox	Records acceptance of the displayed software license agreement.

Security: Do not share ARWeb.lic, response files, machine identifiers, or request payloads in screenshots or support messages unless an authorized secure process requires them.

Part 2 — Bot Job authoring and execution

4.1 Bot Job Details workspace

Entry path: Main Dashboard → select a row → Open Job. Purpose: Author, connect, test, and execute the selected Bot Job.

Figure 08 — Bot Job Details and Execution headers. Screenshot pending.

The first header switches the visible workspace; the second controls execution.

Control	What it does
Refresh	Reloads the current Bot Job graph and owner state.
Bot Job	Returns from another surface to the instruction grid.
Components	Opens the detached Components Library for the selected Bot Job's Organization.
Pre Scan	Opens Page Scanner for the active Bot Job.
Excel	Opens the Bot Job workbook through the configured local action.
Generate	Rebuilds the Bot Job spreadsheet after confirmation.
Report	Chooses and opens an existing report.
Flame-only button	Creates a local BAT launcher.
Starting block	Chooses Execute All or a specific Block.
Reload Blocks	Refreshes the Block selector.
Navigation time	Cycles 0–10 seconds and saves automatically.
ONE	Executes only the selected Block.
ALL ↓	Executes from the selected Block through the remaining Blocks.
Launch	Uses the Excel file as the starting authority, synchronizes Excel Data memory, then executes through Java V1.
Test run	Uses the selected REAL/SYNTHETIC Excel Data memory snapshot, including permitted unsaved test edits.
CHECK	Runs the selected preflight without starting execution.
Stop	Requests execution stop after confirmation.
Export / Import	Transfers the Bot Job to or from a selected folder after confirmation.
Variables	Opens Runtime Variables.
Excel Data	Opens the execution dataset workspace.

Starting Block and ONE/ALL rules

- Execute All always uses ALL and starts at the first Block.
- Selecting a specific Block changes the mode to ONE.
- ONE runs only that selected Block.
- Switch to ALL ↓ to run the selected Block and every later Block in order.

4.2 Execution rules help

Entry path: Bot Job Details → the blue ? between Launch and Test run. Purpose: Explain names, locator priority, Excel Data authority, and execution paths.

Figure 09 — Names, locators, and Excel Data rules. Screenshot pending.

The help tabs are Names & Priority, Excel Data Authority, and Execution Paths. The required locator order is:

- 1 Test ID
- 2 Custom XPath
- 3 XPath
- 4 CSS
- 5 Registry locator
- 6 Canonical name
- 7 `client_named`
- 8 OCR name
- 9 Locator Recovery

Semantic names create candidates; they are not direct browser locators. A candidate still needs uniqueness, visibility, element type/role, and requested-action validation.

4.3 Blocks

Purpose: Group instructions into ordered execution units.

Figure 10 — Block and instruction controls. Screenshot pending.

Block control	What it does
Active/inactive status	Includes or excludes the Block from execution.
Collapse/expand	Hides or shows the instruction rows.
Block checkbox	Selects the Block for the checked-Block delete action.
+	Adds the complete connected Block to Memory List when allowed.
Rollback on first Block	Moves every instruction into the first Block.
Up / Down	Reorders the Block.
Edit	Renames the Block; the confirmed change is saved.
Save/component icon	Creates a reusable component from that Block when available.
Trash	Deletes a Block under the displayed deletion rules.
Selected-row trash	Deletes only checked instructions in that Block after confirmation.

Caution: The first Block has special structural behavior. Read the delete/move confirmation before changing it.

4.4 Instruction rows and web-element actions

An instruction row shows status, selection, command or web-element identity, relationship state, and valid actions.

Row control	What it does
☰	Drags the instruction; keyboard Alt+Arrow can move one position where allowed.
Active/inactive status	Includes or excludes the instruction.
Checkbox	Selects this row for the Block's selected-instruction delete action.
+	Adds the connected instruction group to Memory List.
CLICK / INPUT / OUTPUT	Cycles the web-element execution type and persists the graph-safe replacement.
Wand Review Locator	Opens manual Locator Recovery for this Web Element.
TEST INPUT	Uses the selected Excel Data memory row to test input through the V1 locator resolver.
TEST CLICK	Tests one click through the V1 locator resolver.

Row control	What it does
RENAME	Sets a client-controlled displayed name.
ROLLBACK	Restores the canonical name when a <code>client_named</code> value exists.
EDIT CMD	Opens Command Editor for the selected row.
Up / Down	Reorders the instruction when the relationship graph permits it.
Trash	Deletes the instruction after dependency-aware confirmation.
Tab / Enter	For input elements, configures a follow-up keyboard action.

4.5 Relationships and reconnect dialogs

Purpose: Make required Web Element parents, loop/Block targets, and variables explicit.

Figure 11 — Reconnect Web Element and Reconnect variable. Screenshot pending.

Grey relationship buttons show states such as Reconnect Parent, Reconnect Variable, a connected element ID, or `Variable_name (id: n)`. Click a reconnectable relationship to select a replacement, disconnect it, or—in variable dialogs—use Add Variable. A new independent variable begins as `VOID` and can then be selected for the command.

For check commands, the comparison operator and second-variable control appear in the row. Variables are independent records: deleting an instruction does not silently cascade-delete its variable.

4.6 Command Editor

Entry path: Instruction row → EDIT CMD, or ADD beside Find: to add a disconnected command. Purpose: Configure command type, placement, relationships, and command-specific values.

Figure 12 — Command Editor. Screenshot pending.

- 1 Confirm ADD MODE or EDIT MODE and the displayed Bot Job.
- 2 Choose Target Block and Placement.
- 3 Search and select Command when the instruction type permits it.
- 4 Complete the command-specific editor and variable bindings.
- 5 Use CREATE NEW, COPY NEW, or UPDATE as appropriate.
- 6 Use CANCEL to discard the current editor changes.

Commands include waits, pauses, navigation, loops, GOTO, conditionals, variable operations, checks, and ExcelWrite. Moving a command can display a relationship-impact warning; continue only when the proposed disconnected relationships are understood.

ExcelWrite configuration in Command Editor

An ExcelWrite instruction owns its output file and output column. ExcelWrite file configuration selects directory/file, Excel (.xlsx) or CSV (.csv), and the CSV delimiter. Search existing ExcelWrite files lets multiple instructions or Blocks target the same file when their delimiter contract matches.

4.7 Locator Recovery / Review Locator

Entry path: Web Element row → wand Review Locator; it can also appear automatically when an execution action cannot resolve one valid element. Purpose: Compare the failed Bot Job target with previous database candidates and a fresh current-page scan.

Figure 13 — Locator Recovery. Screenshot pending; this is a priority capture for the client guide.

The first row identifies the Bot Job Web Element that needs recovery. Candidate rows then show saved or current scan evidence. The Origin states are:

Origin	Meaning
BOT JOB	The instruction's current Web Element—the target that failed or was opened manually.
PREVIOUS	A candidate stored from an earlier Page Scanner capture for the owner/page context.
CURRENT	A candidate produced by Page Scanner in the current recovery session.

The table includes Test ID, canonical/client/OCR names, action type, TEST INPUT, TEST CLICK, XPath match, previous/current XPath and CSS, stable attributes, page identities, tag/type/role, confidence, and ambiguity warnings.

Control	What it does
Recovery power toggle	Enables or bypasses locator verification for unresolved elements.
Candidate radio	Chooses the candidate for a recovery decision.
CLICK / INPUT / OUTPUT	Identifies or changes the action used to test that candidate where permitted.
TEST INPUT	Tests input against only the selected candidate, using Excel Data memory.
TEST CLICK	Tests a click against only the selected candidate.
Page Scanner	Runs the normal Page Scanner flow against the current browser page and appends CURRENT candidates.
Close Review	Closes a manually opened review without changing the locator.
Cancel Recovery	Cancels an automatic recovery decision.
Stop Execution	Stops the current execution while automatic recovery is waiting.
Bypass & Continue	Skips the unresolved action and continues.
Use Once	Uses the selected candidate for this occurrence without saving it.
Use and Save Locator	Uses the selected candidate and persists the locator for later runs.
?	Opens Locator Recovery rules and origin explanations.

Caution: A matching name alone is not proof. Prefer Test ID, then stable authored locators. Use and save only after the test action proves the candidate is unique, visible, and correct.

4.8 Components Library

Entry path: Bot Job Details → Components. Purpose: Reuse an Organization's detached instruction and command templates without retaining a live relationship to the source Bot Job.

Figure 13A — Components Library. Screenshot pending; use only the approved synthetic Organization and show one reusable Component with its Memory controls.

Create a Component from a Bot Job

- 1 In Bot Job Details, locate the source Block and click its save/component control.
- 2 In Save component, review the displayed Block number and instruction count.
- 3 Enter Component name and Description.
- 4 Click Save component.
- 5 Open Components and use Refresh if the saved Component is not yet visible.

Saving copies the Block into the Organization's Component Library. The Component is free-standing: its instructions, command configuration, locator references, and private variable copies do not keep a live relationship to the source Bot Job.

Components scope and controls

Rule/control	What it does
Organization scope	Shows only Components owned by the selected Bot Job's Organization. A Component cannot be applied across Organizations.
Refresh	Reloads Components from the authoritative database.
Pages (x)	Opens Pages Open without changing the Component selection.
Close	Closes only the detached Components window.
MEMORY (x)	Opens Memory List with the currently staged Component instructions.
Block +	Adds every instruction in that exact Component Block to Memory List without deleting existing staged items. Repeating the action refreshes matching rows instead of creating duplicates.
Row +	Adds that instruction and its required dependency group when the displayed capability permits it.
CLICK / INPUT / OUTPUT	Changes the reusable Web Element action type.
RENAME / SAVE	Sets and saves a client-controlled name without changing the canonical name.
ROLLBACK	Restores the original canonical name by removing the client-controlled name.

Copy Components into a Bot Job

- 1 Use a Block or row + in Components.
- 2 In Memory List, confirm the source is Reusable Component instructions.
- 3 Select an existing target under Block:, or choose + Create new block....
- 4 Reorder or remove staged rows if necessary.
- 5 Click Apply, or Create & Apply for a new Block.
- 6 Return to Bot Job Details and confirm the copied instructions are independent rows in the target Bot Job.

Caution: Memory List is the only bridge from Components into a Bot Job. Apply fails closed if the Organization, Component revision, target Bot Job, or required relationships changed after staging. Refresh Components and stage the rows again; do not bypass the warning.

Part 3 — Page Scanner, locators, and OCR

5.1 Page Scanner workspace

Entry path: Bot Job Details → Pre Scan. Purpose: Scan the browser page that currently belongs to the selected execution/browser context.

Figure 14 — Page Scanner toolbar. Screenshot pending.

Control	What it does
Page Scanner	Runs the Playwright scanner for the current page and selected focus/search rules.
OCR Config	Opens OCR Configuration.
Refresh Web Page	Reloads the current scanner browser page.
Clear Grid	Clears displayed scanner results, not the webpage.
Starting Block / ONE / ALL ↓ / Test run / Stop	Uses the shared Bot Job execution selection from the Page Scanner workspace.
Focus:	Chooses a reusable scanner profile.
Profile settings	Opens Page Scanner profiles.
Search by:	Accepts selectors or attr:<name> terms such as attr:test-id.
Search Hidden Fields	Includes matching hidden controls in the scan request.
Search	Runs the scan with the current Focus and Search by values.
Find:	Filters results already present in the grid.
Memory (x)	Opens Memory List.
Locator Gen	Opens Locator Generator.

Caution: Page Scanner must target the currently selected V1/browser owner. If the page is still loading, wait for readiness or refresh before concluding that an element is absent.

5.2 Scanner results grid

Purpose: Review, test, rename, retain, and stage scanned Web Elements.

Figure 15 — Page Scanner result Blocks and rows. Screenshot pending.

Grid control	What it does
KEEP ALL	Checks every displayed element as retained.
CLEAR KEEPS	Clears Keep selections.
DELETE UNCHECKED (x)	Deletes displayed elements not marked Keep after confirmation.
CLEAR GRID ALL	Clears the complete displayed result grid.
Rows per page	Selects 5, 10, 20, or 50 rows per scanner Block.
Block collapse	Shows or hides the Block's rows.
Block +	Adds all Web Elements in that Block to Memory List.
ID	Shows the raw DOM ID view.
ID-TEST	Shows test-attribute evidence such as test-id or data-testid.
OCR	Opens the Block's OCR name-review view.
Block TEST INPUT	Types the scanner's test value into input elements in that Block.
Prev / Next	Moves between result pages inside the Block.
Keep checkbox	Protects the row from DELETE UNCHECKED.

Grid control	What it does
Row +	Adds that Web Element to Memory List.
RENAME / ROLLBACK	Sets a client name or restores the original scanned name.
CLICK / INPUT / OUTPUT	Defines the staged Web Element execution type.
TEST INPUT / TEST CLICK	Performs a direct single-shot test using Test ID, XPath, CSS, then reference attributes.
Row X	Removes that result from the current grid.

5.3 Page Scanner profiles

Entry path: Page Scanner → settings beside Focus:. Purpose: Save reusable focus and selector rules.

Figure 16 — Page Scanner profiles. Screenshot pending.

Choose a Profile, edit Name and Order, and add search terms as either Selector or Attribute. The attr: prefix is stored automatically for attribute terms. New starts a profile, Save persists it, Delete removes a non-protected profile, and Refresh search terms from database reloads its rows. The factory default is protected and scans with blank search terms.

5.4 Locator Generator

Entry path: Page Scanner → Locator Gen. Purpose: Convert pasted control HTML into stable ElementDTO locator candidates.

Figure 17 — Locator Generator. Screenshot pending.

- 1 Choose Target scanned element when updating an existing row.
- 2 Paste authorized Control HTML for the colliding controls.
- 3 Click Generate.
- 4 Review generated XPath, CSS, semantic name, and any “positional (fragile)” warning.
- 5 Use Apply XPath to update the selected scanned row, Add ElementDTO for one candidate, or Apply All ElementDTOs for all results.
- 6 Use Clear to reset the pasted HTML and feedback.

5.5 OCR Configuration

Entry path: Page Scanner → OCR Config. Purpose: Select and tune OCR profiles used for scanner name suggestions.

Figure 18 — OCR Configuration. Screenshot pending.

Select Profile, edit Name and Description, and adjust grouped recognition parameters. Test current page tests the active parameter set, Save as new creates a profile, and Save updates it. Delete is disabled for the default profile. Clean orphans removes invalid unowned OCR configuration records according to backend rules.

OCR Block review

On Page Scanner, click OCR on a result Block. Compare each proposed OCR name with the scanned name, accept only correct suggestions, and apply the reviewed names through the displayed review actions. Client-controlled names should be preserved across later scans; use ROLLBACK only when intentionally restoring the canonical/scanned name.

Part 4 — Data, variables, memory, and output files

6.1 Excel Data — execution input memory

Entry path: Bot Job Details → Excel Data. Purpose: Choose and edit the dataset used by Test Run and manual input actions.

Figure 19 — Excel Data in Real Data mode. Screenshot pending.

The header identifies the Bot Job/workbook, connection status, mode, and actions. The metadata line shows file path, mode, selected row, and SAVED or UNSAVED MEMORY. A highlighted cell shows which value is being used during execution or a manual test.

Control	Real Data behavior
Real Data toggle	Switches to Synthetic Data; mode selection is per active Bot Job workspace.
Recreate Columns	Rebuilds workbook columns from Bot Job Blocks and input fields.
Add Row	Copies the last memory row into a new editable test case.
Clean Rows	Removes every memory row while preserving columns.
Save to Excel	Atomically replaces the Bot Job workbook with the current REAL memory rows.
RELOAD FILE	Discards unsaved REAL memory changes and reloads the workbook.
Row X	Deletes one logical row across every Block.
Cell input	Changes only the in-memory value until saved.
?	Opens Excel Data rules.

Real Data authority

- Test run and manual TEST INPUT use Excel Data memory first, including the selected row and permitted unsaved edits.
- Launch reloads the Excel file at startup, updates/replaces Excel Data memory, and runs from that synchronized snapshot.
- A run freezes one mode and row at startup; it must not mix values from different revisions.

Caution: RELOAD FILE discards unsaved REAL memory. Save to Excel makes the memory changes durable in the workbook.

6.2 Excel Data — Synthetic Data mode

Purpose: Create isolated test rows without overwriting the real workbook.

Figure 20 — Excel Data in Synthetic Data mode. Screenshot pending.

Control	Synthetic Data behavior
Synthetic Data toggle	Switches back to Real Data.
Rows	Selects 1–1000 generated test rows.
Context / Search context	Selects a synthetic business-data profile.
Generate Data Test	Replaces synthetic memory with the requested rows after confirmation.
Add Row	Copies the last synthetic memory row.
Clean Rows	Clears synthetic rows while preserving columns.
SAVE DB	Persists the isolated synthetic dataset to SQLite.
Context ?	Explains the available synthetic contexts.

Synthetic rows remain isolated by Bot Job. They never overwrite the real workbook. Selecting Synthetic Data reloads its database dataset automatically.

6.3 ExcelWriter Manager — execution output memory

Entry path: Opened for an execution that contains ExcelWrite; it is also listed in Pages Open. Purpose: Show output files assembled by ExcelWrite instructions before or while they are written to disk.

Figure 21 — ExcelWriter Manager. Screenshot pending.

ExcelWriter Manager is not Excel Data. Excel Data supplies input values; ExcelWriter Manager collects runtime variable values into output files.

Control/state	What it means
Write policy → End of execution	Keeps dirty rows in React memory until the run finishes or another explicit flush boundary occurs.
Write policy → End of each Block	Writes dirty files after each touched Block completes.
Save Dirty Files	Immediately finalizes and writes every dirty file.
File tabs	One tab per output filename; shows row count and MEMORY, UPLOADING, SAVED, or FAILED.
Cell input	Edits the in-memory output cell and marks the file dirty.
File status	Shows final format, revision, and the latest save message.

ExcelWrite file rules

- 1 Each ExcelWrite instruction has an instruction-owned output file and column.
- 2 Instructions targeting the same file share one delimiter.
- 3 A runtime variable in VOID state does not create an ExcelWriter row or file.
- 4 A runtime variable in VALUE state is written even when its value is an empty string or spaces; the resulting cell can be visually empty.
- 5 CSV is always finalized first. A .csv target writes CSV only; a .xlsx target produces the CSV base artifact and the XLSX artifact.
- 6 Stop, authored PAUSE, and authored close-browser (Q/QUIT) are save boundaries for in-progress ExcelWriter data. The stop/cleanup attempt still proceeds if a save reports a failure.
- 7 During an active run, Write policy can be locked so the frozen execution contract cannot change.

Caution: A SAVED tab confirms the output write acknowledged by the application. A FAILED tab must be resolved before assuming a file exists on disk.

6.4 Runtime Variables

Entry path: Bot Job Details → Variables. Purpose: View and edit live Bot Job variable definitions and values.

Figure 22 — Runtime Variables. Screenshot pending.

Control	What it does
Refresh	Reloads current definitions, values, and revisions.
ADD	Opens Add Variable; new independent variables start as VOID.
AUTO	Creates and connects missing command variables under the displayed resolution mode.
CLEAR	Resets all values to VOID without deleting definitions or instruction relationships.
ALL	Requests deletion of all Bot Job variables after confirmation.
Variables search	Filters by variable name or ID.

Control	What it does
Value field	Edits a runtime value; an empty VALUE("") is different from VOID.
Row trash	Deletes that variable after confirmation. Commands and Web Elements remain available.
?	Opens memory-variable rules.

6.5 Memory List

Entry path: Click a row/Block + or Memory (x) in Bot Job, Components, or Page Scanner. Purpose: Stage Bot Job instructions, reusable Component instructions, or scanned Web Elements before applying them to the active Bot Job.

Figure 23 — Memory List. Screenshot pending.

Choose Block: to select a target, or + Create new block.... Drag ≡ to reorder staged items. Row X removes one item, Clear all empties the list, and Apply commits the staged items to the chosen target. Component-sourced rows are copied as independent Bot Job instructions; they do not remain linked to the Component or its source Bot Job. If the staged item set requires a Block and none is selected, Apply remains disabled.

Caution: Memory List is staging. The target Bot Job changes only after Apply succeeds.

6.6 Pages Open

Entry path: Any Pages (x) badge or Main Dashboard user menu → Pages Open. Purpose: Focus and close AR Web pages without guessing which window is behind another.

Figure 24 — Pages Open. Screenshot pending.

Click a page row to bring it to the foreground. Click its X to close that page. Refresh open pages reloads the registry. Closing the Main Dashboard displays Close Main Dashboard? because this closes every AR Web page and stops the backend.

Part 5 — Troubleshooting and quick reference

7.1 A page is disconnected or has no data

- 1 Read the page header status.
- 2 Use its Refresh action once.
- 3 Confirm the correct Bot Job is still selected.
- 4 Open Pages Open and focus the expected page instead of opening duplicates.
- 5 If the Main Dashboard itself has no Bot Jobs, verify that Configuration loaded the intended database.

7.2 Test Run does not start

Run CHECK with the same Starting Block and ONE/ALL mode. Structural failures block startup when the selected graph cannot be executed safely—for example, a required relationship or mandatory configuration is missing. Fix the reported instruction/Block and rerun CHECK.

7.3 An element is not found or is ambiguous

- 1 Wait for the page to finish rendering.
- 2 Confirm that the browser is on the expected page and Bot Job.
- 3 Open Review Locator.
- 4 Inspect the BOT JOB row first.
- 5 Test suitable PREVIOUS candidates.
- 6 Click Page Scanner to add CURRENT candidates.
- 7 Prefer Test ID, then stable authored locators.
- 8 Use Use Once for a one-off recovery or Use and Save Locator only after proof.

7.4 Page Scanner returns PAGE_READINESS_TIMEOUT

The scanner could not prove that the current browser page was ready within its bounded wait. Keep the browser open, allow dynamic content to render, use Refresh Web Page if appropriate, and try once more. Repeated timeouts should be investigated from runtime logs rather than bypassed with coordinates.

7.5 Test Input uses an unexpected value

Open Excel Data and verify:

- the selected mode is Real Data or Synthetic Data as intended;
- the correct row is selected;
- the Block/column name matches the instruction;
- the desired edit is present in memory.

Manual TEST INPUT and Test run use Excel Data memory first. Launch reloads the workbook and may replace unsaved REAL memory.

7.6 ExcelWriter file is missing

Open ExcelWriter Manager and check whether the target tab exists and whether it is MEMORY, SAVED, or FAILED. A missing tab normally means the ExcelWrite instruction was not reached, its file/column is incomplete, or its connected runtime variable remained VOID. Use Save Dirty Files only when a dirty file is visible.

7.7 VOID versus empty value

VOID means no runtime value exists. An empty string is a real VALUE with zero visible characters. This distinction matters:

- ExcelWrite skips VOID.
- ExcelWrite writes an empty cell for VALUE("").
- CLEAR in Runtime Variables changes values to VOID.
- Clearing text in an editable value field can intentionally create an empty value when the page confirms that state.

7.8 Quick control reference

Label	Primary meaning
Refresh	Reload page/workspace data.
Close	Close only that detached page unless the dialog explicitly says application.
Pages (x)	Open the registry of AR Web pages.
Find:	Filter loaded rows locally.
CHECK	Validate the selected execution without starting it.
Launch	File-authoritative normal Java V1 execution.
Test run	Memory-authoritative test execution.
Stop	Request execution stop after confirmation.
TEST INPUT / TEST CLICK	Perform one manual browser action.
Review Locator	Open candidate comparison and locator recovery.
+	Stage a row/Block in Memory List.
Apply	Commit Memory List staging to its target.
Save to Excel	Persist REAL Excel Data memory to the workbook.
SAVE DB	Persist SYNTHETIC Excel Data memory.
Save Dirty Files	Persist dirty ExcelWriter output memory.

7.9 Known documentation limitation

The delivered pages and controls in this edition were verified from the current source branches. The application was running, but the in-app browser-control connection was unavailable and the configured database contains real client data. No screenshots were fabricated or captured from that client dataset. The planned screenshot IDs remain reserved so a safe synthetic session can be captured later without rewriting the chapter structure.

8. Glossary

Term	Meaning
Bot Job	One owned automation definition.
Block	Ordered group of instructions.
Web Element	Browser control targeted by CLICK, INPUT, or OUTPUT behavior.
Test ID	Stable test-oriented HTML attribute used as highest-priority locator evidence.
Canonical name	AR Web internal instruction/element identity.
client_named	Client-controlled alias preserved independently from the canonical name.
OCR name	Name inferred from visible or nearby screen text.
Locator Recovery	Interactive comparison used when a target cannot be uniquely resolved.
Excel Data	Input dataset memory used by tests and synchronized with real or synthetic persistence.
ExcelWriter Manager	Runtime output-file memory populated by ExcelWrite instructions.
VOID	Runtime variable state with no value.
Empty value	A valid value containing zero characters; different from VOID.
Memory List	Staging area for applying instructions/Web Elements to a Bot Job.

9. Source and verification record

Item	Verified value
Documentation repository	ar-web-selenium, branch final-allinweb
Frontend repository	ar-web-allinweb-fe, branch allinweb-delivered, commit 6270db9
Backend/deployment repository	ar-web-allinweb, branch allinweb-delivered, commit 630410a
Delivered runtime	Java V1 execution path; independent React detached workspaces
Screenshot status	Pending safe synthetic capture; no real client data used
Source of truth	This Markdown file